Mini Games: Typing Test, Wordle, Snake, Memory

- **Problem:** Build a small games hub with four keyboard-driven games. A **typing speed test** (30-second timer, live WPM and accuracy, wrong characters highlighted), **Wordle** (guess a 5-letter word in 6 tries, green/yellow/gray feedback, physical and on-screen keyboard), **Snake** (game loop on a grid, arrow keys, food, growth, wall and self collision, pause) and a **memory card game** (flip two cards, match pairs, count moves, win state).
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes (in a real interview you are usually asked for ONE of these; the shared design lets you build any one in 45 minutes)
- **Often asked at:** Microsoft, Uber, Zepto
- **Why it matters:** Game loop, keyboard handling, timers, pure state transitions

---

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, component props, reducer actions.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Idle, running, paused, won, lost, rapid input, tab hidden.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	7 min
Pure logic for the chosen game(s) (reducers, scoring)	15 min
UI + keyboard wiring	18 min
States, edge cases, focus/a11y polish	10 min

Phase	Time
Wrap-up and trade-offs	5 min

If only one game is asked, spend the logic + UI time on it and go deeper on the hard parts (duplicate letters for Wordle, the direction queue for Snake).

Think out loud

Say what you are deciding and why: "I'll put the Snake rules in a pure reducer so the interval never reads stale state, and I can test a tick without a browser." The interviewer grades the reasoning as much as the result. When you take a shortcut, name it.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

Hub

- Switch between the four games. Only one game is active at a time.

Typing test

- Show a random paragraph. The timer (30 s) starts on the first keystroke.
- Each character turns correct (dark) or wrong (red); a caret shows the next character.
- Live time left, WPM and accuracy. Backspace fixes mistakes.
- The test ends when time runs out or the whole text is typed; show the result. Restart gives a new text.

Wordle

- A hidden 5-letter answer, 6 guesses. Type with the physical keyboard or the on-screen one.
- Enter submits: reject short words ("Not enough letters") and unknown words ("Not in word list").
- Each tile becomes green (right letter, right place), yellow (in the word, wrong place) or gray (not in the word), with correct duplicate-letter handling.
- On-screen keys show the best status seen for each letter.
- Win and lose messages (reveal the answer on a loss), New game.

Snake

- A grid with a snake and one food cell. Arrow keys / WASD steer, Space starts and pauses.
- The snake moves on a fixed tick. Eating grows it by one and adds a point; new food never spawns on the snake.
- Hitting a wall or itself ends the game. Play again resets.
- On-screen arrow buttons for touch devices.

Memory

- 16 face-down cards (8 pairs), shuffled. Click to flip.
- Two flipped cards: a match stays face up; a mismatch flips back after a short delay.
- Count moves (one move = two cards). Win message when all pairs are found. Restart reshuffles.

Non-functional requirements

Area	Requirement
Performance	A tick or keystroke does O(grid) work at most; no layout thrash; one interval per running game, never leaked
Correctness	Timer accuracy from timestamps, not from tick counting; game rules testable without a browser
Error handling	Invalid input (wrong word, reversal, clicks while locked, paste in typing test) is ignored or explained, never crashes
Accessibility	Keyboard first; buttons are real `<button>` s with labels; status messages in `aria-live` ; focus is managed so keys reach the game
Reusability	Game rules in pure modules (no React), UI components just render state and dispatch actions
Extensibility	Board size, duration, word list, speed and deck are constants that can become props/levels

Clarifying questions to ask the interviewer

1. Do you want all four games, or should I pick one and go deep? (Usually one; I will design them so they share the same pattern.)
2. Wordle: do guesses need to be real words? Do you have a dictionary, or is a small list fine?
3. Snake: do walls kill, or does the snake wrap around? Should it speed up as it grows?
4. Typing test: fixed time (30 s) or fixed text length? Should WPM count only correct characters?
5. Mobile support: do we need on-screen controls (Wordle keyboard, Snake arrows, tap to flip)?
6. Do we need persistence (best score, Wordle daily word), or is in-memory fine?

Assumptions we will build with

- All four games, in one tabbed hub; only the active game is mounted.
- Wordle uses a small built-in word list that is both the answer pool and the dictionary (named shortcut).
- Snake: 15 x 15 grid, walls kill, speed goes from 150 ms to a 60 ms floor as the score grows. Filling the board is a win.
- Typing test: 30 s or 40 words, whichever ends first. WPM = correct characters / 5 per minute. Paste is blocked.
- Memory: 8 emoji pairs in a 4 x 4 grid; a mismatch is visible for 800 ms.
- No backend and no persistence. Randomness is injected so the logic is deterministic in tests.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Click a game tab	That game mounts fresh; the previous one unmounts (its timers and key listeners are removed); focus moves into the game area
Typing: first character	Timer starts, character turns dark or red, caret moves
Typing: Backspace	Last character removed, its color resets, accuracy recalculates
Typing: paste	Blocked
Typing: 30 s pass or text completed	Input disabled, result panel with WPM, accuracy and errors
Typing: Restart	New text, cleared input, timer back to 30, input focused
Wordle: letter key (physical or on-screen)	Letter added to the current row (max 5)
Wordle: Backspace	Last letter removed
Wordle: Enter with fewer than 5 letters	"Not enough letters", row unchanged
Wordle: Enter with an unknown word	"Not in word list", row unchanged
Wordle: Enter with a valid word	Row is scored green/yellow/gray, keyboard keys recolor, next row active
Wordle: guess equals the answer	"You got it in N!", input locked, New game button
Wordle: 6th wrong guess	"The word was X", input locked, New game button
Wordle: click on-screen key, then press physical Enter	Exactly one submit (the key button never keeps focus)
Snake: Space / arrow key in ready state	Game starts
Snake: arrow key while running	Turn queued; a 180-degree reversal is ignored
Snake: two quick arrows within one tick	Both turns apply on the next two ticks (no lost input, no reversal into the neck)
Snake: head reaches food	Grows by one, score +1, new food on a free cell, slightly faster
Snake: head hits wall or body	"Game over", Play again button
Snake: Space while running / tab hidden	Paused; Space resumes
Memory: click a face-down card	It flips
Memory: second card matches	Both stay up (green), moves +1
Memory: second card does not match	Both stay up 800 ms, then flip back; moves +1

Action / input	Expected behavior / output
Memory: click a third card while two are up	Ignored
Memory: click the same card twice / a matched card	Ignored
Memory: last pair found	"Solved in N moves!", Play again

---

3. Design the architecture

Module	Responsibility
`App`	Game switcher (tabs), mounts only the active game, moves focus into the game area
`TypingTest`	Text, input, timer effect, live stats and result panel
`typingLogic`	`makeText` (random words), `getStats` (WPM, accuracy, errors). Pure
`Wordle`	Renders board + on-screen keyboard, window `keydown` listener, dispatches keys
`wordleLogic`	`evaluateGuess`, `getKeyStatuses`, `wordleReducer`, word list. Pure
`Snake`	Renders grid + controls, interval game loop, keyboard and visibility listeners
`snakeLogic`	`snakeReducer` (turn, tick, toggle, pause, reset), `randomFreeCell`. Pure
`MemoryGame`	Renders the card grid, flip-back timer effect
`memoryLogic`	`shuffle` (Fisher-Yates), `createDeck`, `memoryReducer`. Pure

```
App
 ├── nav (game tabs)
 └── main (focusable, tabIndex=-1)
      └── <ActiveGame key={active} />
           ├── TypingTest ── typingLogic (makeText, getStats)
           ├── Wordle ────── wordleLogic (wordleReducer, evaluateGuess, getKeyStatuses)
           ├── Snake ─────── snakeLogic  (snakeReducer, randomFreeCell)
           └── MemoryGame ── memoryLogic (memoryReducer, createDeck, shuffle)
```

The one pattern every game shares: **pure rules module + thin React component**. The component owns the side effects (timers, listeners, randomness) and turns them into actions; the reducer decides what happens. That is why there is no shared "game engine" abstraction: four reducers are simpler than one generic engine.

---

4. Design the state / data model

Game	State or field	Purpose
App	`active`	Which game is mounted
Typing	`target`	The text to type
Typing	`typed`	What the user typed (controlled input value)
Typing	`status`	`idle` / `running` / `done`
Typing	`startedAt`, `now`	Timestamps; elapsed time is derived from them
Wordle	`answer`	Hidden word
Wordle	`guesses`	Submitted words (strings only)
Wordle	`current`	Letters typed in the active row
Wordle	`status`	`playing` / `won` / `lost`
Wordle	`message`	Validation message ("Not in word list")
Snake	`snake`	Array of `{x, y}`, head first
Snake	`dir`	Current direction
Snake	`queue`	Pending turns (max 2)
Snake	`food`	`{x, y}` or `null` when the board is full
Snake	`score`	Points
Snake	`status`	`ready` / `running` / `paused` / `over` / `won`
Memory	`cards`	`{id, value, matched}[]`
Memory	`flipped`	Ids of face-up unmatched cards (0, 1 or 2)
Memory	`moves`	Pair attempts

```
// Typing: plain useState (few independent fields, simple transitions)
const [target, setTarget] = useState(() => makeText(40));
const [typed, setTyped] = useState('');
const [status, setStatus] = useState('idle');
const [startedAt, setStartedAt] = useState(0);
const [now, setNow] = useState(0);

// Wordle / Snake / Memory: useReducer, because every input is a rule-driven transition
const initWordle = (answer) => ({ answer, guesses: [], current: '', status: 'playing', message: '' });
const initSnake = (r) => ({ snake: [/* 3 cells */], dir: 'right', queue: [], food, score: 0, status:
'ready' });
const initMemory = (cards) => ({ cards, flipped: [], moves: 0 });
```

Senior note: derive, don't store

- **Typing:** `elapsed`, `timeLeft`, `wpm`, `accuracy`, per-character color are all derived from `target`, `typed`, `startedAt`, `now`. Storing `wpm` would let it drift from the text.
- **Wordle:** the tile colors (`results`) and keyboard colors are derived from `guesses` + `answer`. Re-scoring 6 words per render is free and can never be out of sync.
- **Snake:** the cell lookup `Set`, the speed (`delay` from `score`) and the message are derived.
- **Memory:** "board locked" is `flipped.length === 2`, and "won" is `cards.every(c => c.matched)`. No `isLocked` / `isWon` flags to forget to reset.
- **Ownership:** each game owns all its state; `App` owns only `active`. Switching games unmounts the old one, so there is no cross-game state and nothing to clean up by hand.

---

5. Design the data structures and contracts

Example data

```
// Wordle
{ answer: 'APPLE', guesses: ['PAPER'], current: 'SH', status: 'playing', message: '' }
evaluateGuess('PAPER', 'APPLE') // ['present','present','correct','present','absent']

// Snake (15 x 15), head first; cells are {x, y}, flat index = y * SIZE + x
{ snake: [{x:7,y:7},{x:6,y:7},{x:5,y:7}], dir: 'right', queue: ['up'], food: {x:3,y:2}, score: 0,
status: 'running' }

// Memory: id = index after shuffling, so cards[id] is the card (O(1) lookup)
[{ id: 0, value: '🍒', matched: false }, { id: 1, value: '🥝', matched: false }, ...]
```

Why stable ids? Memory cards have duplicate values, so the value cannot be the React key or the identity of a flipped card. The id is assigned once after shuffling and never changes, so it works as the key, as the "flipped" reference and as an array index.

Contracts (reducer actions)

Game	Action	Payload	Effect
Wordle	`key`	`key: 'A'..'Z' | 'ENTER' | 'BACKSPACE'`	Edit row or submit
Wordle	`reset`	`answer`	New game
Snake	`turn`	`dir: 'up' | 'down' | 'left' | 'right'`	Queue a turn (starts the game from ready)
Snake	`tick`	`r: number in [0,1)`	Move one step; `r` picks the next food cell

Game	Action	Payload	Effect
Snake	`toggle` / `pause`	none	Start / pause / resume
Snake	`reset`	`r`	New game
Memory	`flip`	`id`	Flip a card, resolve a pair
Memory	`hide`	none	Flip mismatched cards back
Memory	`reset`	`cards`	New shuffled deck

Randomness enters through actions (`r`, `answer`, `cards`). Reducers stay pure: same state + same action = same result. That makes them testable and safe under React StrictMode, which may call a reducer twice.

There is no API. `getStats(target, typed, elapsedMs) -> { correct, errors, wpm, accuracy }` is the only non-reducer contract that the UI reads.

---

6. Trace the data flow

Snake: the user presses ArrowUp, then the next tick runs

```
keydown "ArrowUp" on window
   │  KEY_TO_DIR["ArrowUp"] = "up";  e.preventDefault() (no page scroll)
   ▼
dispatch({ type: 'turn', dir: 'up' })
   │  snakeReducer: last = queue.at(-1) ?? dir = 'right'
   │  'up' is not 'right' and not OPPOSITE['right'], queue has room
   ▼
state.queue = ['up']            (nothing moves yet)
   ▼
setInterval fires (delay ms)  ──►  dispatch({ type: 'tick', r: Math.random() })
   │  snakeReducer: dir = queue[0] = 'up', head = {x, y - 1}
   │  eats? body = eats ? snake : snake.slice(0, -1)
   │  wall or body hit? -> status 'over'
   │  else snake = [head, ...body]; if eats -> score+1, new food from r
   ▼
new state  ──►  React re-renders the grid (Set lookup per cell)
   ▼
if score changed: delay changes  ──►  effect clears the old interval, starts a faster one
if status became 'over': effect cleanup clears the interval, the loop stops
```

---

7. Important technical challenges

Challenge 1: Wordle duplicate letters

Problem: Answer `APPLE`, guess `PAPER`. A naive "is the letter in the answer" check paints both `P`s yellow or green and also lets one answer letter justify two tiles. Answer `ABBEY`, guess `BOBBY`: the third `B` must be gray because the answer only has two `B`s, and one is already used by the green.

Solution: Two passes. First mark exact matches and count the answer letters that are still unmatched. Second, for each non-green tile, mark it yellow only while unmatched copies remain, decrementing the count.

```
export function evaluateGuess(guess, answer) {
  const result = Array(guess.length).fill('absent');
  const left = {};
  for (let i = 0; i < guess.length; i++) {
    if (guess[i] === answer[i]) result[i] = 'correct';
    else left[answer[i]] = (left[answer[i]] ?? 0) + 1;
  }
  for (let i = 0; i < guess.length; i++) {
    if (result[i] !== 'correct' && left[guess[i]] > 0) {
      result[i] = 'present';
      left[guess[i]]--;
    }
  }
  return result;
}
// BOBBY vs ABBEY -> ['present','absent','correct','absent','correct']
```

Keyboard colors use the same idea of "best status wins": rank `absent < present < correct`, and a key never goes from green back to yellow.

Challenge 2: Snake game loop without stale closures, and the direction queue

Problem A (stale closure): `setInterval(() => move(snake, dir), 150)` captures the `snake` and `dir` of the render that created it. The snake freezes or ignores turns.

Solution: The interval only dispatches `{ type: 'tick' }`. The reducer always receives the latest state, so the closure has nothing to go stale. The effect depends on `[status, delay]`, so it starts on run, stops on pause/game over, and restarts when the speed changes.

```
useEffect(() => {
  if (status !== 'running') return;
  const id = setInterval(() => dispatch({ type: 'tick', r: Math.random() }), delay);
  return () => clearInterval(id);
}, [status, delay]);
```

Problem B (fast double turn): Moving right, the user presses Up then Left within one tick. If we only store a single `dir`, Left overwrites Up and the snake reverses into its own neck (Left is the opposite of Right). If we validate against the current `dir` only, Down after Up is accepted and kills the snake.

Solution: A small queue of pending turns (max 2). Each new turn is validated against the **last queued** direction, and each tick consumes one.

```
case 'turn': {
  const last = state.queue.at(-1) ?? state.dir;
  if (state.queue.length >= 2 || action.dir === last || action.dir === OPPOSITE[last]) return state;
  return { ...state, queue: [...state.queue, action.dir], status: 'running' };
}
```

Problem C (tail chase): The head moving into the cell the tail is leaving is legal, because the tail moves this tick. Checking collision against the full body wrongly ends the game.

```
const eats = same(head, state.food);
const body = eats ? state.snake : state.snake.slice(0, -1); // the tail leaves unless we grow
if (hitWall || body.some((c) => same(c, head))) return { ...state, status: 'over' };
```

Food placement: Picking random cells until one is free gets slow (and loops forever) as the board fills. Collect the free cells and pick one; `null` means the board is full, which is a win.

Challenge 3: Keyboard focus (one key, two handlers)

Problem: Window `keydown` listeners and focused buttons both react to Enter and Space. Three real bugs come from this:

1. Click the "Wordle" tab, type a word, press Enter: the focused tab button gets "clicked" again and the guess never submits.
2. Click the on-screen `ENTER` key, then press the physical Enter: the guess submits twice.
3. Click Snake's Start button, then press Space: the window handler toggles AND the focused button clicks, so the game pauses and unpauses.

Solution: Three small rules.

```
// 1. App: switching games moves focus into the game area (main has tabIndex={-1})
function open(id) { setActive(id); mainRef.current.focus(); }

// 2. On-screen keys never take focus
<button onMouseDown={(e) => e.preventDefault()} onClick={() => dispatch({ type: 'key', key })} />

// 3. Enter/Space on a focused button belongs to the button (keyboard users tabbing to it)
if (e.target.closest?.('button') && (e.key === 'Enter' || e.key === ' ')) return;
```

Also: ignore keys with Ctrl/Meta/Alt (Ctrl+R must still reload), and `preventDefault` arrows and Space in Snake so the page does not scroll.

Challenge 4: Timers that are accurate and always cleaned up

Problem: Counting `setInterval` ticks to measure time drifts (intervals are throttled to 1 per second or worse in background tabs). A Memory flip-back `setTimeout` that fires after the user pressed Restart flips cards of the new deck. Switching games with a running interval leaks it.

Solution:

- Typing time is `Date.now() - startedAt`, never tick counting. The interval only refreshes `now` for re-rendering.
- Every timer lives in a `useEffect` with a cleanup. The Memory timer depends on `[flipped]`, so a reset (new `flipped` array) cancels it.
- `App` mounts only the active game with `key={active}`, so leaving a game unmounts it and runs every cleanup.
- Snake pauses on `visibilitychange` so it does not die in a background tab.

```
useEffect(() => {
  if (flipped.length !== 2) return;
  const id = setTimeout(() => dispatch({ type: 'hide' }), FLIP_BACK_MS);
  return () => clearTimeout(id);
}, [flipped]);
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	`App` with tabs and `key={active}` mounting	Can switch between four placeholder games
2	Memory: `shuffle`, `createDeck`, render 16 buttons	Shuffled grid, cards show values
3	Memory: `memoryReducer` flip/match + flip-back effect	Playable memory game with moves and win
4	Typing: render target + controlled input + per-char colors	Colors and caret follow typing
5	Typing: timer effect + `getStats` + done state + restart	Full typing test
6	Wordle: `evaluateGuess` (two-pass) with a few asserts	Correct scoring incl. duplicates
7	Wordle: reducer + board render + window keydown	Playable with physical keyboard
8	Wordle: on-screen keyboard with key colors, messages, new game	Full Wordle
9	Snake: grid render + `snakeReducer` tick + interval loop	Snake moves and dies at the wall
10	Snake: turn queue, food, growth, self collision, speed	Full Snake
11	Snake: pause, visibility pause, on-screen arrows	Usable on mobile and in background tabs

#	Step	Working result after the step
12	Focus fixes, `aria-live` messages, node assert self-check of all logic	Robust and verified

9. Cover all states and edge cases

Game	State / edge case	Handling
Typing	Idle (not started)	Timer shows 30 s, WPM 0, accuracy 100%, placeholder hint
Typing	Running	Interval refreshes `now` every 200 ms
Typing	First second	WPM shown as 0 (3 chars in 50 ms would be 720 WPM)
Typing	Text finished before time	Ends immediately; WPM uses the real elapsed time
Typing	Time runs out	Elapsed clamped to 30 s; input disabled
Typing	Typing past the text / paste	Value sliced to target length; paste blocked
Wordle	Short word / unknown word	Message, row kept
Wordle	Typing a 6th letter	Ignored
Wordle	Duplicate letters	Two-pass scoring
Wordle	Won / lost	Input locked by the reducer; lost reveals the answer
Wordle	Modifier shortcuts (Ctrl+R)	Not intercepted
Snake	Ready	Message; first Space or valid arrow starts
Snake	Reversal / third quick turn	Ignored
Snake	Tail chase	Legal
Snake	Paused / tab hidden	No interval; turns ignored
Snake	Board full	`food = null`, status `won`
Snake	Wall / self hit	`over`, Play again
Memory	Same card clicked twice	Ignored
Memory	Third card while two are up	Ignored (locked)
Memory	Matched card	Disabled button
Memory	Restart during the 800 ms delay	Effect cleanup cancels the old timer
Memory	All matched	Win message, Play again
All	Switching games mid-play	Unmount runs every cleanup; new game starts fresh

10. Final structure

```
src/
├── main.jsx
├── index.css                # @import "tailwindcss"
├── App.jsx                  # tabs, active game, focus move
├── logic.test.mjs           # node assert self-check of all pure logic
└── games/
    ├── typing/
    │   ├── typingLogic.js   # makeText, getStats
    │   └── TypingTest.jsx
    ├── wordle/
    │   ├── wordleLogic.js   # evaluateGuess, getKeyStatuses, wordleReducer, WORDS
    │   └── Wordle.jsx
    ├── snake/
    │   ├── snakeLogic.js    # snakeReducer, randomFreeCell, DIRS, KEY_TO_DIR
    │   └── Snake.jsx
    └── memory/
        ├── memoryLogic.js   # shuffle, createDeck, memoryReducer
        └── MemoryGame.jsx
```

---

11. Full working code

Setup: `npm create vite@latest games -- --template react`, then `npm i -D tailwindcss @tailwindcss/vite` and add `tailwindcss()` to the Vite plugins.

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/index.css

```
@import "tailwindcss";

body {
  @apply bg-slate-50 text-slate-900;
}
```

src/App.jsx

```
import { useRef, useState } from 'react';
import TypingTest from './games/typing/TypingTest';
import Wordle from './games/wordle/Wordle';
import Snake from './games/snake/Snake';
import MemoryGame from './games/memory/MemoryGame';

const GAMES = [
  { id: 'typing', label: 'Typing test', Component: TypingTest },
  { id: 'wordle', label: 'Wordle', Component: Wordle },
  { id: 'snake', label: 'Snake', Component: Snake },
  { id: 'memory', label: 'Memory', Component: MemoryGame },
];

export default function App() {
  const [active, setActive] = useState('typing');
  const { Component } = GAMES.find((g) => g.id === active);
  const mainRef = useRef(null);

  function open(id) {
    setActive(id);
    // move focus off the tab button, or the next Enter would "click" the tab instead of reaching the
game
    mainRef.current.focus();
  }

  return (
    <div className="mx-auto max-w-2xl px-4 py-8">
      <h1 className="mb-4 text-2xl font-bold">Mini Games</h1>
```

```
      <nav className="mb-6 flex flex-wrap gap-2" aria-label="Choose a game">
        {GAMES.map((g) => (
          <button
            key={g.id}
            onClick={() => open(g.id)}
            aria-pressed={g.id === active}
            className={`rounded-full px-4 py-1.5 text-sm ${g.id === active ? 'bg-sky-600 text-white' :
'bg-slate-200'}`}
          >
            {g.label}
          </button>
        ))}
      </nav>
      {/* only the active game is mounted: its timers and window key listeners die with it */}
      <main ref={mainRef} tabIndex={-1} className="outline-none">
        <Component key={active} />
      </main>
    </div>
  );
}
```

src/games/typing/typingLogic.js

```
export const WORDS = `the be of and a to in he have it that for they with as not on she at by this we
you do but
from or which one would all will there say who make go when can more if no man out other so what time
up about
than into could state only new year some take come these know see use get like then first any work now
may such
give over think most where after back`.split(/\s+/);

// rand is injectable so tests get a fixed text
export function makeText(count, rand = Math.random) {
  return Array.from({ length: count }, () => WORDS[Math.floor(rand() * WORDS.length)]).join(' ');
}

// WPM = (correct chars / 5) per minute; "5 chars = 1 word" is the standard definition
export function getStats(target, typed, elapsedMs) {
  let correct = 0;
  for (let i = 0; i < typed.length; i++) if (typed[i] === target[i]) correct++;
  const minutes = elapsedMs / 60000;
  return {
    correct,
    errors: typed.length - correct,
    // under 1s the number explodes (3 chars in 50ms = 720 WPM), so wait a second before showing it
    wpm: elapsedMs < 1000 ? 0 : Math.round(correct / 5 / minutes),
```

```
    accuracy: typed.length ? Math.round((correct / typed.length) * 100) : 100,
  };
}
```

src/games/typing/TypingTest.jsx

```
import { useEffect, useRef, useState } from 'react';
import { getStats, makeText } from './typingLogic';

const DURATION = 30; // seconds
const WORD_COUNT = 40;

export default function TypingTest() {
  const [target, setTarget] = useState(() => makeText(WORD_COUNT));
  const [typed, setTyped] = useState('');
  const [status, setStatus] = useState('idle'); // idle | running | done
  const [startedAt, setStartedAt] = useState(0);
  const [now, setNow] = useState(0);
  const inputRef = useRef(null);

  // the clock only runs while the test runs; cleanup kills it on finish, restart and unmount
  useEffect(() => {
    if (status !== 'running') return;
    const id = setInterval(() => {
      const t = Date.now();
      setNow(t);
      if (t - startedAt >= DURATION * 1000) setStatus('done');
    }, 200);
    return () => clearInterval(id);
  }, [status, startedAt]);

  // time comes from timestamps, not from counting ticks, so a throttled tab stays accurate
  const elapsed = status === 'idle' ? 0 : Math.min(now - startedAt, DURATION * 1000);
  const timeLeft = Math.ceil(DURATION - elapsed / 1000);
  const stats = getStats(target, typed, elapsed);

  function handleChange(e) {
    if (status === 'done') return;
    const value = e.target.value.slice(0, target.length);
    const t = Date.now();
    if (status === 'idle') {
      setStartedAt(t);
      setStatus('running');
    }
    setNow(t);
    setTyped(value);
```

```
    if (value.length === target.length) setStatus('done');
  }

  function restart() {
    setTarget(makeText(WORD_COUNT));
    setTyped('');
    setStatus('idle');
    inputRef.current?.focus();
  }

  return (
    <section className="space-y-4">
      <div className="flex gap-6 text-sm text-slate-600" aria-live="polite">
        <span>Time: <b className="text-slate-900">{timeLeft}s</b></span>
        <span>WPM: <b className="text-slate-900">{stats.wpm}</b></span>
        <span>Accuracy: <b className="text-slate-900">{stats.accuracy}%</b></span>
      </div>

      <p
        className="cursor-text rounded-lg border border-slate-200 bg-white p-4 font-mono text-lg
leading-8"
        onClick={() => inputRef.current?.focus()}
      >
        {target.split('').map((ch, i) => {
          let cls = 'text-slate-400';
          if (i < typed.length) cls = typed[i] === ch ? 'text-slate-900' : 'bg-rose-100 text-rose-600';
          if (i === typed.length && status !== 'done') cls += ' border-l-2 border-sky-500';
          return <span key={i} className={cls}>{ch}</span>;
        })}
      </p>

      <input
        ref={inputRef}
        value={typed}
        onChange={handleChange}
        onPaste={(e) => e.preventDefault()}
        disabled={status === 'done'}
        autoFocus
        autoComplete="off"
        autoCapitalize="off"
        spellCheck={false}
        aria-label="Type the text shown above"
        placeholder={status === 'idle' ? 'Start typing to begin the timer…' : ''}
        className="w-full rounded-lg border border-slate-300 px-3 py-2 font-mono outline-none
focus:border-sky-500"
      />
```

```
      {status === 'done' && (
        <div className="rounded-lg bg-sky-50 p-4" role="status">
          <p className="font-semibold">
            {stats.wpm} WPM · {stats.accuracy}% accuracy · {stats.errors} errors
          </p>
        </div>
      )}
      <button onClick={restart} className="rounded-lg bg-slate-900 px-4 py-2 text-white">
        Restart
      </button>
    </section>
  );
}
```

src/games/wordle/wordleLogic.js

```
export const WORD_LENGTH = 5;
export const MAX_GUESSES = 6;

// shortcut: one small list is both the answer pool and the dictionary; load a real word list for
production
export const WORDS = `APPLE BRAVE CRANE DRIVE EAGLE FLAME GRAPE HOUSE IVORY JOKER KNIFE LEMON MANGO
NIGHT OCEAN
PIANO QUEEN RIVER SNAKE TIGER VIVID WATER YOUTH ZEBRA ABOUT ALERT BEACH BLAME BRAIN BREAD CHAIR CHEST
CLOUD
DANCE EARTH FAITH FIELD FRUIT GHOST GLASS HEART HORSE LIGHT MONEY MUSIC NORTH PAPER PARTY PLANT POWER
RADIO
SCALE SHARP SMILE SOUND SPACE STONE SUGAR TABLE TEACH THINK TRAIN TRUST WHEEL WORLD WRITE ABBEY LLAMA
SPOON
SHEEP GRASS`.split(/\s+/);
const WORD_SET = new Set(WORDS);

export const randomWord = (rand = Math.random) => WORDS[Math.floor(rand() * WORDS.length)];

// Two passes so duplicate letters are counted right:
// 1) mark exact hits and count the answer letters that are still unmatched
// 2) a letter is "present" only while unmatched copies of it are left
export function evaluateGuess(guess, answer) {
  const result = Array(guess.length).fill('absent');
  const left = {};
  for (let i = 0; i < guess.length; i++) {
    if (guess[i] === answer[i]) result[i] = 'correct';
    else left[answer[i]] = (left[answer[i]] ?? 0) + 1;
  }
  for (let i = 0; i < guess.length; i++) {
    if (result[i] !== 'correct' && left[guess[i]] > 0) {
```

```
      result[i] = 'present';
      left[guess[i]]--;
    }
  }
  return result;
}

const RANK = { absent: 1, present: 2, correct: 3 };

// on-screen keyboard colors: a key keeps its best status ever (green never goes back to yellow)
export function getKeyStatuses(guesses, results) {
  const map = {};
  guesses.forEach((guess, row) =>
    [...guess].forEach((ch, i) => {
      const s = results[row][i];
      if (!map[ch] || RANK[s] > RANK[map[ch]]) map[ch] = s;
    }),
  );
  return map;
}

export const initWordle = (answer) => ({ answer, guesses: [], current: '', status: 'playing', message:
'' });

// keys: 'A'..'Z', 'ENTER', 'BACKSPACE'. Pure, so the physical and on-screen keyboards share one path.
export function wordleReducer(state, action) {
  if (action.type === 'reset') return initWordle(action.answer);
  if (state.status !== 'playing') return state;
  const { key } = action;
  if (key === 'BACKSPACE') return { ...state, current: state.current.slice(0, -1), message: '' };
  if (key === 'ENTER') {
    if (state.current.length < WORD_LENGTH) return { ...state, message: 'Not enough letters' };
    if (!WORD_SET.has(state.current)) return { ...state, message: 'Not in word list' };
    const guesses = [...state.guesses, state.current];
    const status =
      state.current === state.answer ? 'won' : guesses.length === MAX_GUESSES ? 'lost' : 'playing';
    return { ...state, guesses, current: '', status, message: '' };
  }
  if (/^[A-Z]$/.test(key) && state.current.length < WORD_LENGTH) {
    return { ...state, current: state.current + key, message: '' };
  }
  return state;
}
```

`src/games/wordle/Wordle.jsx`

```
import { useEffect, useReducer } from 'react';
import {
  MAX_GUESSES, WORD_LENGTH, evaluateGuess, getKeyStatuses, initWordle, randomWord, wordleReducer,
} from './wordleLogic';

const KEY_ROWS = ['QWERTYUIOP', 'ASDFGHJKL', ['ENTER', ...'ZXCVBNM', 'BACKSPACE']];
const COLORS = {
  correct: 'bg-emerald-600 border-emerald-600 text-white',
  present: 'bg-amber-500 border-amber-500 text-white',
  absent: 'bg-slate-500 border-slate-500 text-white',
};

export default function Wordle() {
  const [state, dispatch] = useReducer(wordleReducer, undefined, () => initWordle(randomWord()));
  const { guesses, current, status, answer, message } = state;

  // derived, never stored: re-scoring at most 6 words per render costs nothing
  const results = guesses.map((g) => evaluateGuess(g, answer));
  const keyStatus = getKeyStatuses(guesses, results);

  // dispatch is stable, so the listener is added once and never reads stale state
  useEffect(() => {
    function onKeyDown(e) {
      if (e.ctrlKey || e.metaKey || e.altKey) return;
      // Enter/Space on a focused button belongs to that button (New game, a tabbed-to key)
      if (e.target.closest?.('button') && (e.key === 'Enter' || e.key === ' ')) return;
      const key = e.key.toUpperCase();
      if (key !== 'ENTER' && key !== 'BACKSPACE' && !/^[A-Z]$/.test(key)) return;
      e.preventDefault();
      dispatch({ type: 'key', key });
    }
    window.addEventListener('keydown', onKeyDown);
    return () => window.removeEventListener('keydown', onKeyDown);
  }, []);

  const rows = Array.from({ length: MAX_GUESSES }, (_, r) => {
    if (r < guesses.length) return { letters: guesses[r], result: results[r] };
    if (r === guesses.length) return { letters: current, result: null };
    return { letters: '', result: null };
  });

  return (
    <section className="flex flex-col items-center gap-4">
      <div className="grid gap-1.5" role="grid" aria-label="Wordle board">
        {rows.map((row, r) => (
```

```
    <div key={r} className="grid grid-cols-5 gap-1.5" role="row">
      {Array.from({ length: WORD_LENGTH }, (_, i) => {
        const ch = row.letters[i] ?? '';
        const s = row.result?.[i];
        return (
          <div
            key={i}
            role="gridcell"
            aria-label={s ? `${ch} ${s}` : ch || 'empty'}
            className={`flex size-12 items-center justify-center border-2 text-xl font-bold
              ${s ? COLORS[s] : ch ? 'border-slate-500' : 'border-slate-300'}`}
          >
            {ch}
          </div>
        );
      })}
    </div>
  ))}
</div>

<p className="h-6 text-sm font-medium" aria-live="polite">
  {status === 'won' && `You got it in ${guesses.length}!`}
  {status === 'lost' && `The word was ${answer}`}
  {status === 'playing' && message}
</p>

{status !== 'playing' && (
  <button
    onClick={() => dispatch({ type: 'reset', answer: randomWord() })}
    className="rounded-lg bg-slate-900 px-4 py-2 text-white"
  >
    New game
  </button>
)}

<div className="flex flex-col items-center gap-1.5">
  {KEY_ROWS.map((row, r) => (
    <div key={r} className="flex gap-1">
      {[...row].map((key) => (
        <button
          key={key}
          // keep focus off the key, otherwise a later physical Enter would "click" it again
          onMouseDown={(e) => e.preventDefault()}
          onClick={() => dispatch({ type: 'key', key })}
          aria-label={key === 'BACKSPACE' ? 'Backspace' : key === 'ENTER' ? 'Enter' : key}
          className={`h-12 rounded font-semibold ${key.length > 1 ? 'px-2 text-xs' : 'w-8'}
            ${COLORS[keyStatus[key]] ?? 'bg-slate-200'}`}
```

```
              >
                {key === 'BACKSPACE' ? '⌫' : key}
              </button>
            ))}
          </div>
        ))}
      </div>
    </section>
  );
}
```

src/games/snake/snakeLogic.js

```
export const SIZE = 15;
export const DIRS = {
  up: { x: 0, y: -1 },
  down: { x: 0, y: 1 },
  left: { x: -1, y: 0 },
  right: { x: 1, y: 0 },
};
const OPPOSITE = { up: 'down', down: 'up', left: 'right', right: 'left' };
export const KEY_TO_DIR = {
  ArrowUp: 'up', w: 'up', ArrowDown: 'down', s: 'down',
  ArrowLeft: 'left', a: 'left', ArrowRight: 'right', d: 'right',
};

const same = (a, b) => a.x === b.x && a.y === b.y;

// r is a number in [0, 1) passed in by the caller, so the reducer stays pure and tests are
deterministic
export function randomFreeCell(snake, r, size = SIZE) {
  const taken = new Set(snake.map((c) => c.y * size + c.x));
  const free = [];
  for (let i = 0; i < size * size; i++) if (!taken.has(i)) free.push(i);
  if (!free.length) return null;
  const i = free[Math.floor(r * free.length)];
  return { x: i % size, y: Math.floor(i / size) };
}

export function initSnake(r) {
  const mid = Math.floor(SIZE / 2);
  const snake = [{ x: mid, y: mid }, { x: mid - 1, y: mid }, { x: mid - 2, y: mid }]; // head first
  return { snake, dir: 'right', queue: [], food: randomFreeCell(snake, r), score: 0, status: 'ready' };
}

// status: ready | running | paused | over | won
```

```
export function snakeReducer(state, action) {
  switch (action.type) {
    case 'turn': {
      if (state.status !== 'running' && state.status !== 'ready') return state;
      // compare with the last QUEUED turn, not the current dir: "up, left" fast from "right" is legal,
      // "up, down" inside one tick is a reversal into our own neck
      const last = state.queue.at(-1) ?? state.dir;
      if (state.queue.length >= 2 || action.dir === last || action.dir === OPPOSITE[last]) return
state;
      return { ...state, queue: [...state.queue, action.dir], status: 'running' };
    }
    case 'toggle':
      if (state.status === 'running') return { ...state, status: 'paused' };
      if (state.status === 'paused' || state.status === 'ready') return { ...state, status: 'running'
};
      return state;
    case 'pause':
      return state.status === 'running' ? { ...state, status: 'paused' } : state;
    case 'reset':
      return initSnake(action.r);
    case 'tick': {
      if (state.status !== 'running') return state;
      const [next, ...queue] = state.queue;
      const dir = next ?? state.dir;
      const head = { x: state.snake[0].x + DIRS[dir].x, y: state.snake[0].y + DIRS[dir].y };
      const eats = same(head, state.food);
      // the tail moves away this tick unless we eat, so stepping into the old tail cell is legal
      const body = eats ? state.snake : state.snake.slice(0, -1);
      const hitWall = head.x < 0 || head.y < 0 || head.x >= SIZE || head.y >= SIZE;
      if (hitWall || body.some((c) => same(c, head))) return { ...state, dir, queue, status: 'over' };
      const snake = [head, ...body];
      if (!eats) return { ...state, snake, dir, queue };
      const food = randomFreeCell(snake, action.r);
      return { ...state, snake, dir, queue, food, score: state.score + 1, status: food ? 'running' :
'won' };
    }
    default:
      return state;
  }
}
```

src/games/snake/Snake.jsx

```
import { useEffect, useReducer } from 'react';
import { KEY_TO_DIR, SIZE, initSnake, snakeReducer } from './snakeLogic';
```

```
const MESSAGES = {
  ready: 'Press Space or an arrow key to start',
  paused: 'Paused: press Space to resume',
  over: 'Game over',
  won: 'You filled the board!',
};

export default function Snake() {
  const [state, dispatch] = useReducer(snakeReducer, undefined, () => initSnake(Math.random()));
  const { snake, food, score, status } = state;
  const delay = Math.max(60, 150 - score * 5); // speeds up as you eat

  // game loop: one interval while running; the reducer always sees fresh state, so no stale closure
  useEffect(() => {
    if (status !== 'running') return;
    const id = setInterval(() => dispatch({ type: 'tick', r: Math.random() }), delay);
    return () => clearInterval(id);
  }, [status, delay]);

  useEffect(() => {
    function onKeyDown(e) {
      if (e.target.closest?.('button') && (e.key === 'Enter' || e.key === ' ')) return;
      const dir = KEY_TO_DIR[e.key.length === 1 ? e.key.toLowerCase() : e.key];
      if (dir) {
        e.preventDefault(); // arrows would scroll the page
        dispatch({ type: 'turn', dir });
      } else if (e.key === ' ') {
        e.preventDefault();
        dispatch({ type: 'toggle' });
      }
    }
    const onVisibility = () => document.hidden && dispatch({ type: 'pause' });
    window.addEventListener('keydown', onKeyDown);
    document.addEventListener('visibilitychange', onVisibility);
    return () => {
      window.removeEventListener('keydown', onKeyDown);
      document.removeEventListener('visibilitychange', onVisibility);
    };
  }, []);

  // O(1) lookup per cell instead of snake.some() inside a 225-cell loop
  const body = new Set(snake.map((c) => c.y * SIZE + c.x));
  const head = snake[0].y * SIZE + snake[0].x;
  const foodIdx = food ? food.y * SIZE + food.x : -1;

  return (
    <section className="flex flex-col items-center gap-4">
```

```
      <div className="flex gap-6 text-sm text-slate-600">
        <span>Score: <b className="text-slate-900">{score}</b></span>
        <span aria-live="polite">{MESSAGES[status] ?? ''}</span>
      </div>

      <div
        role="img"
        aria-label={`Snake board, length ${snake.length}, score ${score}`}
        className="grid aspect-square w-full max-w-[360px] gap-px rounded bg-slate-200 p-px"
        style={{ gridTemplateColumns: `repeat(${SIZE}, 1fr)` }}
      >
        {Array.from({ length: SIZE * SIZE }, (_, i) => (
          <div
            key={i}
            className={
              i === head ? 'bg-emerald-700'
                : body.has(i) ? 'bg-emerald-500'
                : i === foodIdx ? 'rounded-full bg-rose-500'
                : 'bg-white'
            }
          />
        ))}
      </div>

      <div className="grid grid-cols-3 gap-1" aria-label="Direction controls">
        {[['', ''], ['up', '↑'], ['', ''], ['left', '←'], ['down', '↓'], ['right', '→']].map(([dir,
label], i) =>
          dir ? (
            <button
              key={i}
              aria-label={`Move ${dir}`}
              onMouseDown={(e) => e.preventDefault()} // keep focus free so Space still pauses
              onClick={() => dispatch({ type: 'turn', dir })}
              className="size-12 rounded bg-slate-200 text-xl"
            >
              {label}
            </button>
          ) : (
            <span key={i} />
          ),
        )}
      </div>

      <div className="flex gap-2">
        {(status === 'over' || status === 'won') ? (
          <button
            onClick={() => dispatch({ type: 'reset', r: Math.random() })}
```

```
            className="rounded-lg bg-slate-900 px-4 py-2 text-white"
          >
            Play again
          </button>
        ) : (
          <button onClick={() => dispatch({ type: 'toggle' })} className="rounded-lg bg-slate-900 px-4
py-2 text-white">
            {status === 'running' ? 'Pause' : 'Start'}
          </button>
        )}
      </div>
    </section>
  );
}
```

src/games/memory/memoryLogic.js

```
export const SYMBOLS = ['🍎', '🍌', '🍇', '🍒', '🥝', '🍊', '🍍', '🥥'];

// Fisher-Yates: unbiased, O(n). (sort(() => Math.random() - 0.5) is biased.)
export function shuffle(arr, rand = Math.random) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(rand() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

// ids are given AFTER shuffling and the array never reorders, so cards[id] is the card
export const createDeck = (rand = Math.random) =>
  shuffle([...SYMBOLS, ...SYMBOLS], rand).map((value, id) => ({ id, value, matched: false }));

export const initMemory = (cards = createDeck()) => ({ cards, flipped: [], moves: 0 });

export function memoryReducer(state, action) {
  switch (action.type) {
    case 'flip': {
      const card = state.cards[action.id];
      // board is "locked" while two cards are showing: derived from flipped.length, no extra flag
      if (state.flipped.length === 2 || card.matched || state.flipped.includes(card.id)) return state;
      if (!state.flipped.length) return { ...state, flipped: [card.id] };
      const first = state.cards[state.flipped[0]];
      const moves = state.moves + 1;
      if (first.value !== card.value) return { ...state, flipped: [first.id, card.id], moves };
      return {
```

```
        ...state,
        moves,
        flipped: [],
        cards: state.cards.map((c) => (c.id === first.id || c.id === card.id ? { ...c, matched: true }
: c)),
      };
    }
    case 'hide':
      return { ...state, flipped: [] };
    case 'reset':
      return initMemory(action.cards);
    default:
      return state;
  }
}
```

src/games/memory/MemoryGame.jsx

```
import { useEffect, useReducer } from 'react';
import { createDeck, initMemory, memoryReducer } from './memoryLogic';

const FLIP_BACK_MS = 800;

export default function MemoryGame() {
  const [state, dispatch] = useReducer(memoryReducer, undefined, () => initMemory());
  const { cards, flipped, moves } = state;
  const won = cards.every((c) => c.matched);

  // a mismatch stays visible for a moment; cleanup cancels the timer on reset/unmount
  useEffect(() => {
    if (flipped.length !== 2) return;
    const id = setTimeout(() => dispatch({ type: 'hide' }), FLIP_BACK_MS);
    return () => clearTimeout(id);
  }, [flipped]);

  return (
    <section className="flex flex-col items-center gap-4">
      <p className="text-sm text-slate-600" aria-live="polite">
        Moves: <b className="text-slate-900">{moves}</b>
        {won && <span className="ml-4 font-semibold text-emerald-700">Solved in {moves} moves!</span>}
      </p>

      <div className="grid grid-cols-4 gap-2">
        {cards.map((card) => {
          const faceUp = card.matched || flipped.includes(card.id);
          return (
```

```
            <button
              key={card.id}
              onClick={() => dispatch({ type: 'flip', id: card.id })}
              disabled={card.matched}
              aria-label={faceUp ? `Card ${card.value}` : 'Hidden card'}
              className={`flex size-16 items-center justify-center rounded-lg text-3xl transition-
colors
                ${card.matched ? 'bg-emerald-100' : faceUp ? 'bg-white ring-2 ring-sky-400' : 'bg-
slate-700 hover:bg-slate-600'}`}
            >
              {faceUp ? card.value : ''}
            </button>
          );
        })}
      </div>

      <button
        onClick={() => dispatch({ type: 'reset', cards: createDeck() })}
        className="rounded-lg bg-slate-900 px-4 py-2 text-white"
      >
        {won ? 'Play again' : 'Restart'}
      </button>
    </section>
  );
}
```

`src/logic.test.mjs` (run with `node src/logic.test.mjs`)

```
// Self-check of the pure game logic: node src/logic.test.mjs
import assert from 'node:assert/strict';
import { getStats, makeText } from './games/typing/typingLogic.js';
import { evaluateGuess, getKeyStatuses, initWordle, wordleReducer } from
'./games/wordle/wordleLogic.js';
import { initSnake, randomFreeCell, snakeReducer } from './games/snake/snakeLogic.js';
import { createDeck, initMemory, memoryReducer } from './games/memory/memoryLogic.js';

// typing
assert.equal(makeText(3, () => 0), 'the the the');
assert.deepEqual(getStats('hello world', 'hellx', 60000), { correct: 4, errors: 1, wpm: 1, accuracy: 80
});
assert.equal(getStats('abc', 'ab', 500).wpm, 0); // no WPM before 1s
assert.equal(getStats('abc', '', 0).accuracy, 100);

// wordle: duplicate letters
assert.deepEqual(evaluateGuess('PAPER', 'APPLE'), ['present', 'present', 'correct', 'present',
'absent']);
```

```
assert.deepEqual(evaluateGuess('BOBBY', 'ABBEY'), ['present', 'absent', 'correct', 'absent',
'correct']);
assert.deepEqual(evaluateGuess('SHEEP', 'SHEEP'), Array(5).fill('correct'));
const keys = getKeyStatuses(['PAPER', 'APPLE'], [evaluateGuess('PAPER', 'APPLE'),
evaluateGuess('APPLE', 'APPLE')]);
assert.equal(keys.A, 'correct');
assert.equal(keys.R, 'absent');
const typeWord = (s, w) => [...w, 'ENTER'].reduce((st, key) => wordleReducer(st, { type: 'key', key }),
s);
let w = initWordle('APPLE');
assert.equal(typeWord(w, 'APP').message, 'Not enough letters');
assert.equal(typeWord(w, 'XXXXX').message, 'Not in word list');
assert.equal(wordleReducer(w, { type: 'key', key: 'BACKSPACE' }).current, '');
w = typeWord(w, 'PAPER');
assert.equal(w.guesses.length, 1);
assert.equal(typeWord(w, 'APPLE').status, 'won');
for (let i = 0; i < 6; i++) w = typeWord(w, 'GRAPE');
assert.equal(w.status, 'lost');
assert.equal(typeWord(w, 'APPLE').guesses.length, 6); // input ignored after game over

// snake
let s = initSnake(0);
assert.equal(s.status, 'ready');
assert.deepEqual(s.food, { x: 0, y: 0 });
s = snakeReducer(s, { type: 'turn', dir: 'left' }); // reversal ignored
assert.equal(s.status, 'ready');
s = snakeReducer(s, { type: 'turn', dir: 'up' });
s = snakeReducer(s, { type: 'turn', dir: 'down' }); // reversal of the queued turn ignored
assert.deepEqual(s.queue, ['up']);
s = snakeReducer(s, { type: 'tick', r: 0 });
assert.deepEqual(s.snake[0], { x: 7, y: 6 });
assert.equal(s.snake.length, 3);
// eat
s = { ...s, food: { x: 7, y: 5 } };
s = snakeReducer(s, { type: 'tick', r: 0 });
assert.equal(s.score, 1);
assert.equal(s.snake.length, 4);
// wall
let wall = { snake: [{ x: 0, y: 3 }, { x: 1, y: 3 }], dir: 'left', queue: [], food: { x: 9, y: 9 },
score: 0, status: 'running' };
assert.equal(snakeReducer(wall, { type: 'tick', r: 0 }).status, 'over');
// chasing the tail is legal (tail leaves this tick)
const loop = {
  snake: [{ x: 0, y: 0 }, { x: 1, y: 0 }, { x: 1, y: 1 }, { x: 0, y: 1 }],
  dir: 'left', queue: ['down'], food: { x: 9, y: 9 }, score: 0, status: 'running',
};
assert.equal(snakeReducer(loop, { type: 'tick', r: 0 }).status, 'running');
```

```
// self hit
const self = { ...loop, snake: [...loop.snake, { x: 0, y: 2 }] };
assert.equal(snakeReducer(self, { type: 'tick', r: 0 }).status, 'over');
assert.equal(randomFreeCell([{ x: 0, y: 0 }], 0, 2).x, 1);
assert.equal(randomFreeCell([{ x: 0, y: 0 }, { x: 1, y: 0 }, { x: 0, y: 1 }, { x: 1, y: 1 }], 0.5, 2),
null);
// pause / toggle
assert.equal(snakeReducer({ ...loop }, { type: 'pause' }).status, 'paused');
assert.equal(snakeReducer({ ...loop, status: 'paused' }, { type: 'tick', r: 0 }).status, 'paused');

// memory
const deck = createDeck(() => 0);
assert.equal(deck.length, 16);
assert.ok(deck.every((c, i) => c.id === i));
let m = initMemory(deck);
const [a, b] = deck.filter((c) => c.value === deck[0].value);
const other = deck.find((c) => c.value !== deck[0].value);
m = memoryReducer(m, { type: 'flip', id: a.id });
m = memoryReducer(m, { type: 'flip', id: a.id }); // same card twice ignored
assert.deepEqual(m.flipped, [a.id]);
m = memoryReducer(m, { type: 'flip', id: b.id });
assert.ok(m.cards[a.id].matched && m.cards[b.id].matched);
assert.equal(m.moves, 1);
m = memoryReducer(m, { type: 'flip', id: other.id });
const third = deck.find((c) => c.value !== other.value && !m.cards[c.id].matched);
m = memoryReducer(m, { type: 'flip', id: third.id });
assert.equal(m.flipped.length, 2); // mismatch stays up
const fourth = deck.find((c) => !m.flipped.includes(c.id) && !m.cards[c.id].matched);
assert.equal(memoryReducer(m, { type: 'flip', id: fourth.id }), m); // locked while 2 are up
assert.equal(memoryReducer(m, { type: 'hide' }).flipped.length, 0);

console.log('all logic checks passed');
```

---

12. Wrap-up

Edge cases handled

- Wordle duplicate letters (two-pass), invalid and short words, input after game over.
- Snake reversal, fast double turns (queue), tail chase, food never on the snake, full board, pause on hidden tab, no page scroll on arrows.
- Memory third-click lock, double click on the same card, timer cancelled on restart.
- Typing test timer from timestamps, first-second WPM spike, overflow and paste.
- Focus: tab buttons and on-screen keys never swallow Enter/Space meant for the game.

Trade-offs to mention

Decision	Why	Cost
`useReducer` with pure rules	No stale closures, unit-testable, StrictMode-safe	Slightly more boilerplate than `useState`
DOM grid for Snake (225 divs)	Simple, accessible, styled with CSS	A 100 x 100 board would need `<canvas>`
`setInterval` for the snake	Fixed tick is the game rule; easy to pause	Not frame-synced; `requestAnimationFrame` + accumulator gives smoother animation
Typing accuracy from the current text	Simple and honest after corrections	Does not count corrected mistakes (raw keystroke accuracy needs a keystroke counter)
Small word list as dictionary	No network, instant	Real Wordle needs ~13k valid guesses (a `Set`, loaded lazily)
One component per game, no shared engine	Four small reducers beat one generic abstraction	Some repeated keydown wiring

What to add with more time

- Tests: React Testing Library for keyboard flows (Wordle Enter, Snake pause), fake timers for Memory and Typing.
- Persistence: best scores and Wordle daily word (seeded by date) in `localStorage`.
- Wordle tile flip animation and row shake on invalid word; Memory 3D flip with CSS `transform`.
- Snake on `<canvas>` with `requestAnimationFrame`, wrap-around mode, swipe gestures.
- Difficulty levels (grid size, deck size, typing duration) as props.
- Lazy-load each game with `React.lazy` so the hub ships only the active game.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why build all four with the same pattern instead of four different styles?

Because the interviewer usually picks one, and the pattern (pure reducer + thin component + effects for timers and listeners) works for all of them. It also lets me reuse the same testing approach and the same focus rules.

Q2. What if the interviewer wants a "daily" Wordle like the real one?

Pick the answer from the date instead of `Math.random`: `WORDS[daysSinceEpoch % WORDS.length]`, and persist the guesses for that date in `localStorage` so a reload keeps the board. The reducer does not change, only the `answer` passed into `reset`.

State design

Q3. Why `useReducer` for Wordle, Snake and Memory but `useState` for Typing?

The three games have rule-driven transitions where the next state depends on several fields at once (current row + guesses + status, snake + queue + food). A reducer keeps them atomic and testable. The typing test only sets a few independent values from one handler, so `useState` is enough.

Q4. Why not store the tile colors in Wordle state?

They are a pure function of `guesses` and `answer`. Storing them creates a second source of truth that can go out of sync (for example on reset). Re-scoring at most 6 words per render costs nothing.

Q5. Why is the random number passed in the action instead of calling `Math.random()` inside the reducer?

Reducers must be pure. React StrictMode can call them twice, and tests need deterministic results. The component (which owns side effects) generates `r` and the reducer just uses it, so `snakeReducer(state, {type:'tick', r: 0})` always gives the same answer.

Challenge: Wordle scoring

Q6. Walk me through `evaluateGuess('BOBBY', 'ABBEY')`.

Pass 1: position 2 (`B`) and 4 (`Y`) are green; unmatched answer letters are `A, B, E` (one `B`). Pass 2: position 0 `B` takes the one remaining `B` (yellow), position 1 `O` is gray, position 3 `B` finds no `B` left (gray). Result: present, absent, correct, absent, correct.

Q7. What if the word list is 13,000 words?

Use a `Set` for O(1) "is this a word" checks (already done), load the list lazily (dynamic `import()` or `fetch` when Wordle opens) so it does not bloat the main bundle, and keep the answer list separate and smaller than the valid-guess list.

Challenge: Snake loop

Q8. Why not `setInterval(() => setSnake(move(snake)), 150)`?

The callback captures `snake` from the render that created the interval, so every tick moves the same old snake. You can fix it with functional updates, but direction, food and score also need fresh values, so dispatching `tick` to a reducer is cleaner: the reducer always gets the latest state.

Q9. Why a queue of turns and not just the latest direction?

Two quick presses within one tick (Up then Left while moving Right) would otherwise lose the first turn and, worse, Left would reverse into the neck. The queue keeps both, validates each against the previous queued direction, and is capped at 2 so mashing keys does not build a backlog.

Q10. `setInterval` or `requestAnimationFrame`?

Snake is a fixed-step game, so a fixed interval matches the rule and is easy to pause. For smooth animation I would use `requestAnimationFrame` with an accumulator: add the frame delta, and run `tick` each time the accumulator passes the step size. rAF also pauses automatically in background tabs.

Q11. What if the grid is 100 x 100?

10,000 DOM cells re-rendering every tick is too much. Switch to `<canvas>` and draw only what changed (erase the old tail cell, draw the new head and food). Collision stays O(1) with a `Set` of occupied cells kept alongside the array.

Challenge: focus and keyboard

Q12. Why does Enter sometimes not submit the Wordle guess, and how did you fix it?

After clicking a button (the game tab or an on-screen key) that button keeps focus, and the browser turns Enter into a click on it. I move focus to the game container when a game opens, stop on-screen keys from taking focus with `onMouseDown preventDefault`, and let a focused button keep its own Enter/Space for keyboard users.

Q13. Why listen on `window` and not on the board element?

The board is not an input, so it only gets key events when focused. Listening on `window` works no matter where the user clicked. The cost is that two mounted games would fight over keys, which is why only the active game is mounted and its listener is removed on unmount.

Challenge: timers

Q14. How is the typing timer accurate in a background tab?

The interval only triggers re-renders. Time is always `Date.now() - startedAt`, so if the browser throttles the interval to once per second, the next tick still shows the correct time and ends the test at the right moment.

Q15. What happens if the user presses Restart in Memory while a mismatch is showing?

`reset` replaces `flipped` with a new empty array, so the effect cleanup runs and clears the pending `setTimeout`. Without the cleanup, the old timer would fire and hide cards of the new deck.

Implementation steps

Q16. Why Fisher-Yates and not `array.sort(() => Math.random() - 0.5)`?

The sort trick is biased (some orders are far more likely, depending on the engine's sort algorithm) and is O(n log n). Fisher-Yates gives every permutation equal probability in O(n).

Q17. Why is the card id its index after shuffling?

Values are duplicated, so they cannot identify a card. Assigning ids after shuffling and never reordering the array makes `cards[id]` an O(1) lookup and gives React a stable key.

Performance

Q18. Typing test with a 5,000-character text: what gets slow?

Every keystroke re-renders 5,000 spans. Split the text into words and memoize each word component (`React.memo`) so only the word under the caret re-renders, or render only a window of lines around the caret.

Q19. What if the API for words is slow?

Show a loading state, keep the built-in list as a fallback so the game is always playable, and cache the fetched list. The reducer does not care where the answer comes from.

Trade-offs

Q20. Why no shared `useGameLoop` or `useKeyboard` hook?

Each game uses its loop and keys differently (Snake interval, Memory one-shot timeout, Typing clock, Wordle no timer). A shared hook would be a thin wrapper around `useEffect` with options for each case. I would extract one when a fifth game needs the same thing.

Q21. How would you make Snake and Wordle accessible to screen reader users?

Status changes (score, game over, "Not in word list") are announced with `aria-live`. Wordle tiles have labels like "P absent". Snake is inherently visual; I would add a text summary ("Head at row 3, food 2 up, 4 left") on demand and allow a slower speed.